Solving Sudoku the Hard Way Using Integer-Valued p-adic Linear Regression with Positive and Negative Regularisation

(Author details omitted for review)

March 11, 2026

Abstract

This note studies a general p-adic encoding of finite-domain all-different systems and then uses Sudoku as a concrete case study. Given a graph whose vertices carry finite allowed sets and whose edges reward unequal assignments, we show that a p-adic regression-style objective has global minimisers exactly the domain-respecting assignments that maximise the number of unequal edges. When the corresponding list-colouring instance is satisfiable, the global minimisers are precisely the proper list-colourings. Sudoku fits this template by taking one variable per cell, edges between cells that share a row, column, or box, and singleton domains on clue cells. To make the regression viewpoint explicit, we expand the unary penalties into synthetic observations that "snap" each coefficient to a digit and use negative regularisation on pairwise differences to reward inequality. For primes $p > 9$, the p-adic norm makes every nonzero digit difference have norm 1, eliminating magnitude bias and turning "difference" into a clean proxy for "not equal". We then "train" the resulting regression instance using two generic optimisers: a stepwise/coordinate-descent heuristic (row-wise swap moves) and a Zubarev-style stochastic walk that samples updates with probability proportional to $\exp(-\beta\,\Delta L)$. Despite the layered absurdity—solving Sudoku by first forcing it into a p-adic regression mould and then applying generic regression training rules—both heuristics can solve moderate-clue puzzles in small-scale experiments.

1 Motivation and prior work

There are many sensible ways to solve Sudoku puzzles. Exact-cover formulations solved by Knuth's Algorithm X with dancing links [3], constraint-programming formulations built around *alldifferent* constraints [4], and SAT encodings [5] are all natural approaches. This paper is not one of them.

Instead, it asks whether modifying the loss landscape of linear regression is sufficient to encode finite-domain all-different systems using digit-valued parameters directly rather than one-hot indicators, with Sudoku as the cleanest worked example. The point is not computational efficiency but conceptual compression: use fewer variables than a typical SAT/CSP encoding, and let the p-adic objective enforce both domain membership and pairwise distinctness.

Two earlier results motivate the construction:

- A structural result on p-adic linear regression [1]: an optimal hyperplane is supported by $n + 1$ data points.
- A complexity result on p-adic linear regression [2]: the problem is NP-hard, via an embedding of Max-Cut using regularisation points that snap coefficients to $\{0, 1\}$.

The central proposition below isolates the general mechanism behind the Sudoku construction.

2 Preliminaries

2.1 p-adic integers and norms

Fix a prime p. The p-adic valuation $v_p(n)$ of an integer $n \neq 0$ is the exponent of p in its factorisation, and the p-adic absolute value is

$$|n|_p = p^{-v_p(n)}, \qquad |0|_p = 0.$$

We will work in the p-adic integers $\mathbb{Z}_p$, where all norms satisfy $|x|_p \leq 1$. The key choice is to take $p > 9$ (e.g. $p = 11$).

3 A general all-different regression template

Many finite-domain constraint systems built from all-different conditions can be viewed as list-colouring problems on a graph. Each variable x_i is assigned a value from a finite allowed set D_i, and an edge $\{i, j\}$ expresses the preference or requirement that $x_i \neq x_j$. Over $\mathbb{Z}_p$ with $p > q$, the p-adic norm is perfectly suited to this setting: once values are restricted to $\{1, \ldots, q\}$, every nonzero difference has norm 1. The next proposition packages that observation into a clean optimisation statement.

Proposition 1 (Finite-domain all-different systems as p-adic regression)**.** *Let $V = \{1, \ldots, n\}$, let $G = (V, E)$ be a finite graph, let $D_i \subseteq \{1, \ldots, q\}$ be nonempty allowed sets, and fix a prime $p > q$. For each $i \in V$, choose once and for all a reference value $a_i^* \in D_i$, and define the coordinatewise snapping map $\pi_i : \mathbb{Z}_p \to D_i$ by*

$$\pi_i(t) := \begin{cases} t, & t \in D_i, \\ a, & t \notin D_i \text{ and } t \equiv a \pmod{p} \text{ for the unique } a \in D_i, \\ a_i^*, & t \not\equiv a \pmod{p} \text{ for all } a \in D_i. \end{cases}$$

Since $p > q$, the residues of $1, \ldots, q$ are distinct modulo p, so the middle case is well defined. Let

$$\Pi : \mathbb{Z}_p^n \to \prod_{i=1}^{n} D_i, \qquad \Pi(x) := (\pi_1(x_1), \ldots, \pi_n(x_n)).$$

Also define

$$w_{ii} := 0, \qquad w_{ij} := \begin{cases} 1, & \{i, j\} \in E, \\ 0, & \{i, j\} \notin E, \end{cases} \qquad (i \neq j).$$

Fix $\beta > 0$, and set

$$\lambda := 1 + \beta \max_{i \in V} \sum_{j=1}^{n} w_{ij}.$$

For $t \in \mathbb{Z}_p$, define

$$R_i(t) := \sum_{a \in D_i} |t - a|_p - (|D_i| - 1).$$

Consider

$$L(x) := \lambda \sum_{i=1}^{n} R_i(x_i) - \beta \sum_{1 \leq i < j \leq n} w_{ij} |x_i - x_j|_p.$$

If $x \in \mathbb{Z}_p^n$ and $i \in V$ with $x_i \notin D_i$, let $x^{(i)}$ be the vector obtained from x by replacing x_i with $\Pi(x)_i$. Then

$$L(x^{(i)}) < L(x).$$

Consequently every global minimiser of L lies in $\prod_{i=1}^n D_i$. On $\prod_{i=1}^n D_i$,

$$L(x) = -\beta \sum_{1\le i<j\le n} w_{ij}\mathbf{1}_{x_i\neq x_j}.$$

Consequently, the global minimisers are exactly the domain-respecting assignments that maximise the number of unequal edges. In particular, if the list-colouring instance $(G,(D_i)_{i=1}^n)$ is satisfiable, then the global minimisers are exactly its proper list-colourings.

Proof. For every $i \in V$ and $t \in \mathbb{Z}_p$, we have

$$R_i(t) = |t - \pi_i(t)|_p$$

Indeed, if $t \in D_i$ then both sides are 0. If $t \notin D_i$ but $t \equiv a \pmod p$ for the unique $a \in D_i$, then $|t-a|_p < 1$, while for every $b \in D_i \setminus \{a\}$ we have $t-b \equiv a-b \not\equiv 0 \pmod p$, so $|t-b|_p = 1$. Thus $R_i(t) = |t-a|_p = |t-\pi_i(t)|_p$. If t is congruent modulo p to no element of D_i, then $|t-a|_p = 1$ for all $a \in D_i$, hence $R_i(t) = 1 = |t-a_i^*|_p = |t-\pi_i(t)|_p$.

Now fix $x \in \mathbb{Z}_p^n$ and $i \in V$ with $x_i \notin D_i$. Put $a := \Pi(x)_i = \pi_i(x_i)$. Since $a \in D_i$, we have $R_i(a) = 0$, so the unary part changes by

$$\lambda\big(R_i(a) - R_i(x_i)\big) = -\lambda|x_i - a|_p.$$

For each $j \in V$, the corresponding interaction changes by

$$\beta w_{ij}\big(|x_i - x_j|_p - |a - x_j|_p\big).$$

By the ultrametric inequality,

$$|x_i - x_j|_p \le \max\big(|x_i - a|_p,\ |a - x_j|_p\big),$$

so

$$|x_i - x_j|_p - |a - x_j|_p \le |x_i - a|_p.$$

Summing over all $j \in V$,

$$L(x^{(i)}) - L(x) \le \left(-\lambda + \beta\sum_{j=1}^n w_{ij}\right)|x_i - a|_p \le -|x_i - a|_p < 0.$$

Therefore every out-of-domain coordinate can be snapped into D_i with a strict loss decrease. In particular, no global minimiser can have $x_i \notin D_i$, and every global minimiser lies in $\prod_{i=1}^n D_i$.

If $x_i \in D_i \subseteq \{1,\dots,q\}$ for all i, then $R_i(x_i) = 0$ for each i. Moreover, for $1 \le i < j \le n$,

$$|x_i - x_j|_p = \begin{cases} 0, & x_i = x_j, \\ 1, & x_i \neq x_j, \end{cases}$$

because whenever $x_i \neq x_j$ we have $1 \le |x_i - x_j| \le q-1 < p$. This yields

$$L(x) = -\beta \sum_{1\le i<j\le n} w_{ij}\mathbf{1}_{x_i\neq x_j}.$$

The final claims follow immediately. □

The proposition separates the mechanism cleanly into two parts. First, the map Π strictly lowers the loss whenever some variable lies outside its prescribed finite domain. Once every variable has been pinned into $\prod_{i=1}^n D_i$, the pairwise p-adic terms collapse to indicator functions for inequality. What remains is exactly an all-different or list-colouring objective.

Corollary 2. *Any finite-domain all-different constraint system can be encoded as the global minimiser set of a p-adic regression objective.*

Proof. Given an all-different constraint system, form its primal graph: the vertices are the variables, and two variables are adjacent if they appear together in some all-different constraint. The satisfying assignments are exactly the proper list-colourings of this graph, with the variable domains as lists. Apply Proposition 1. □

4 Sudoku as a special case

4.1 Sudoku

A (standard) Sudoku puzzle consists of a partially filled 9×9 grid. The task is to assign a digit in $\{1, \ldots, 9\}$ to each empty cell such that:

1. each row contains all digits $1, \ldots, 9$ exactly once;
2. each column contains all digits $1, \ldots, 9$ exactly once; and
3. each 3×3 box contains all digits $1, \ldots, 9$ exactly once.

Equivalently, in each unit (row, column, box) all entries are *pairwise distinct* and each entry lies in $\{1, \ldots, 9\}$.

Standard Sudoku fits Corollary 2 in the most literal possible way. The vertices are the 81 cells, two vertices are adjacent when the corresponding cells share a row, column, or 3×3 box, unclued cells have domain $\{1, \ldots, 9\}$, and clue cells have singleton domains. A proper list-colouring of this graph is exactly a valid Sudoku completion.

The objective used in the experiments below is a deliberately expanded version of this general construction. Instead of keeping one compact unary penalty R_i per variable, it exposes the individual synthetic residuals that build those penalties. This makes the regression interpretation more transparent and is the form most convenient for counting residuals and describing the heuristics.

4.2 Positive regularisation: snapping coefficients to digits

Let $D = \{1, 2, \ldots, 9\} \subset \mathbb{Z}_p$, and write the Sudoku variables as a vector

$$x \in \mathbb{Z}_p^{81},$$

with one coefficient per cell.

Definition 1 (Digit-snapping penalty)**.** *For $p > 9$, define the digit-snapping penalty*

$$R_p(t) := \sum_{k=1}^{9} |t - k|_p.$$

Lemma 1 (Snapping to D for $p > 9$)**.** *Assume $p > 9$ and $t \in \mathbb{Z}_p$. Then $R_p(t)$ is minimised precisely when $t \in D$. Moreover, $R_p(k) = 8$ for $k \in D$ and $R_p(t) > 8$ for $t \notin D$.*

Proof. If $t = k \in D$, then the kth term is $|k - k|_p = 0$ and for any $k' \neq k$ we have $1 \leq |k - k'| \leq 8 < p$, so $p \nmid (k - k')$ and hence $|k - k'|_p = 1$. Thus $R_p(k) = 8$.

Now take $t \notin D$. If $t \not\equiv k \pmod{p}$ for every $k \in D$, then $p \nmid (t - k)$ for all k, so $|t - k|_p = 1$ for all k and $R_p(t) = 9 > 8$. Otherwise, $t \equiv k_0 \pmod{p}$ for exactly one $k_0 \in D$ (distinct digits differ by $< p$ so the residue classes are distinct). Then $|t - k_0|_p = p^{-m}$ for some $m \geq 1$, while for $k \neq k_0$ we still have $|t - k|_p = 1$. Hence $R_p(t) = 8 + p^{-m} > 8$. □

This lemma is the 9-valued analogue of the earlier "snap to $\{0, 1\}$" construction from [2]: the synthetic observations $t = k$ for all $k \in D$ create a multiwell objective with minima exactly at D. In the language of Proposition 1, it is the uniform-domain case $D_i = D$.

4.3 Negative regularisation: rewarding inequality via p-adic norms

The remaining Sudoku constraints are that entries in each row/column/box are pairwise unequal. Pairwise inequality can be encoded using differences $x_i - x_j$.

Lemma 2 (Differences as inequality indicators for $p > 9$)**.** *Let $p > 9$ and $a, b \in D$. Then*

$$|a - b|_p = \begin{cases} 0 & a = b, \\ 1 & a \neq b. \end{cases}$$

Proof. If $a = b$ the difference is zero. If $a \neq b$, then $1 \leq |a - b| \leq 8 < p$, so $p \nmid (a - b)$ and $v_p(a - b) = 0$, hence $|a - b|_p = 1$. □

4.4 Expanded regression objective

Let $\mathcal{P}$ be the set of cell-pairs (i, j) that share a unit (same row, same column, or same box). Let $\mathcal{C}$ be the set of given clues; for $i \in \mathcal{C}$ let $g_i \in D$ be the fixed digit.

We construct a linear regression objective by treating each constraint as a synthetic observation of a linear form $a^\top x$ with target b and p-adic residual norm:

$$L(x) = \alpha \sum_{i=1}^{81} \sum_{k=1}^{9} |x_i - k|_p \; + \; \alpha_{\text{clue}} \sum_{i \in \mathcal{C}} |x_i - g_i|_p \; - \; \beta \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p. \tag{1}$$

Equation (1) can be read as a regression problem with an explicit set of synthetic observations: 81 multiwell "snap to digit" terms (implemented as $81 \cdot 9 = 729$ residuals $x_i - k$), $|\mathcal{C}|$ clue residuals $x_i - g_i$, and $|\mathcal{P}| = 810$ pairwise-difference residuals $x_i - x_j$ (each cell has 20 peers, counted once). Equivalently, counting pairs per unit gives $27\binom{9}{2} = 972$ terms, but 162 of these pairs lie simultaneously in a row/column *and* a box, leaving 810 distinct pairs overall.

The first two sums are *positive* regularisation that encourage $x_i \in D$ and enforce the clues. The final sum is *negative* regularisation that rewards inequality: for digit-valued assignments it contributes $-\beta$ for every pair of unequal entries and 0 for equal entries. The 810 pairwise terms are essentially irreducible if one insists on *pairwise* penalties: omitting a single peer-pair (i, j) removes the only residual that directly distinguishes $x_i = x_j$ from $x_i \neq x_j$ once x is digit-valued, and no linear "hash" of the remaining differences can substitute because nonvanishing is not preserved under linear combinations (e.g. $a - b \neq 0$ and $b - c \neq 0$ does not prevent $a - c = 0$).

The compact objective in Proposition 1 would use one domain penalty R_i per cell, with singleton domains on clues. Equation (1) is a more redundant but regression-friendly expansion of the same idea: it keeps the 729 digit-snapping residuals visible and writes the clue penalties separately rather than folding them into the domains. This is not the tersest way to state the theorem, but it is the most convenient form for implementation and for counting the synthetic observations explicitly.

Corollary 3 (Sudoku completion as a special case)**.** *Let G_{Sud} be the graph on the 81 cells of a Sudoku puzzle, with an edge between two cells exactly when they share a row, column, or 3×3 box. For a puzzle with clue set $\mathcal{C}$ and clue digits g_i, define*

$$D_i := \begin{cases} \{g_i\}, & i \in \mathcal{C}, \\ \{1, \ldots, 9\}, & i \notin \mathcal{C}. \end{cases}$$

Then the global minimisers of the objective from Proposition 1 are exactly the valid completions of the Sudoku puzzle.

Proof. Each row, column, and box of Sudoku forms a clique of size 9 in G_{Sud}. Thus a proper list-colouring assigns pairwise distinct digits in each unit. Since the available values are exactly $\{1, \ldots, 9\}$, pairwise distinctness in a unit is equivalent to containing every digit exactly once. The singleton domains enforce the clues. Apply Proposition 1. □

Algorithm 1 Row-swap stepwise regression (heuristic)

Require: Puzzle with clues $\mathcal{C}$, max steps T, restarts R

```
for r = 1 to R do
    Initialise each row as a permutation of 1, ..., 9 consistent with its clues
    for t = 1 to T do
        if no column/box conflicts then
            return solution
        end if
        Choose a row involved in a conflict
        Find the best swap of two non-clue cells in that row (largest loss decrease)
        Apply the swap (or a random swap if no improving swap exists)
    end for
end for
return best assignment found
```

For Sudoku, this bookkeeping is completely explicit. In the current draft, $\mathcal{P}$ is the set of *distinct* peer pairs, so each cell has exactly 20 incident negative terms. In the notation of Proposition 1, this means $\sum_j w_{ij} = 20$ for every cell, so one may take $\lambda = 1 + 20\beta$. In the expanded objective (1), the same domination condition appears as $\alpha > 20\beta$ (for example $\alpha = 1 + 20\beta$). If instead one rewrote the loss as a sum over row, column, and box scopes and counted overlaps separately, each cell would contribute $8 + 8 + 8 = 24$ incident terms, and the corresponding sufficient choice would be $\alpha > 24\beta$. This is the precise place where the negative regularisation has to be dominated by the positive unary weight.

Why p-adics help with negative weights. If one tries to do this with an ordinary squared-loss term $-(x_i - x_j)^2$ in $\mathbb{R}$, the optimiser is perversely incentivised to make some differences *large*, not merely nonzero, and the objective can become unstable unless heavily bounded. With $p > 9$ and digit-valued variables, $|x_i - x_j|_p \in \{0, 1\}$, removing magnitude bias.

5 A stepwise/coordinate-descent heuristic

The objective (1) induces an NP-hard global optimisation problem (consistent with the author's NP-hardness result for p-adic regression). The goal here is not a polynomial-time solver, but a proof-of-concept heuristic.

In implementation we take the "$\alpha \to \infty$" limit by restricting search to digit-valued assignments consistent with clues. Concretely, we initialise each row as a random permutation of $1, \ldots, 9$ consistent with its clues, and then apply row-wise swaps that reduce the remaining loss (column/box conflicts). This is a form of coordinate descent on an integer lattice.

This "stepwise regression" view matches the informal idea: at each step, choose the coefficient change that gives the largest incremental loss improvement.

5.1 A short trace on a standard puzzle

To make the row-swap heuristic more concrete, Figure 1 shows the standard example puzzle. With seed 0 (restart 0), the row-wise random initialisation used by the solver produces the filled grid in Figure 2; this state has 56 column+box conflict pairs. Table 1 shows the first few swaps from that initial state, and the algorithm occasionally applies a random swap when no improving move is available, to escape local minima.

For this instance the solver reaches a full solution after 228 swaps on the same restart.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Figure 1: Standard example puzzle used for the trace.

5	3	8	4	7	2	1	9	6
6	2	8	1	9	5	3	4	7
3	9	8	1	4	7	2	6	5
8	2	9	5	6	4	7	1	3
4	5	7	8	9	3	2	6	1
7	3	1	8	2	5	4	9	6
3	6	1	7	9	5	2	8	4
2	6	3	4	1	9	7	8	5
3	2	4	6	8	5	1	7	9

Figure 2: Seed-0 row-wise random initialisation (restart 0), before any swaps.

6 A Zubarev-style stochastic walk

The "stepwise regression" heuristic above is tailored to the strong-snapping Sudoku setting: evaluate all within-row swaps and greedily choose the best improving move (or a random move when stuck). Once we take seriously the idea that Sudoku is being treated as a regression instance, it is natural—if delightfully over the top—to also try a generic training method from the p-adic regression literature.

6.1 Zubarev's proposal in the regression setting

In [7], Zubarev notes that loss functions built from sums of p-adic norms are locally constant almost everywhere, so gradient methods are uninformative. He therefore suggests a stochastic alternative: "we can propose a stochastic optimization algorithm for the function (9) based on a random walk in the parameter space w" [7].

Concretely, the proposed training dynamics is a discrete-time random process

$$w_{t+1} = w_t + \xi_t(w_t, \beta_t), \tag{2}$$

where $\beta_t > 0$ is a "continuity" (inverse temperature) parameter and ξ_t is sampled from a distribution biased toward loss-decreasing moves. In Zubarev's formulation the conditional law

Step	Row	Swap (columns)	Type	Conflicts (before)	(after)
1	5	$7 \leftrightarrow 8$	best	56	53
2	6	$3 \leftrightarrow 6$	best	53	49
3	2	$2 \leftrightarrow 3$	best	49	45
4	4	$7 \leftrightarrow 6$	random	45	47
5	7	$1 \leftrightarrow 5$	best	47	40
6	4	$6 \leftrightarrow 7$	best	40	38
7	2	$2 \leftrightarrow 9$	best	38	36
8	1	$3 \leftrightarrow 4$	best	36	35

Table 1: First eight swap steps for the example puzzle (seed 0). Rows/columns are 1-indexed.

has density (with respect to the p-adic Haar measure)

$$P(\xi \mid w, \beta) \;\propto\; \exp\bigl(-\beta\,(L(w+\xi) - L(w))\bigr). \tag{3}$$

Under mild conditions (in particular, that the set of improving moves has nonzero measure), Zubarev shows there is a regime of β values for which the expected loss decreases, i.e. $L(w_t)$ is a strict supermartingale [7].

6.2 A discrete Sudoku analogue

In our "$\alpha \to \infty$" implementation, the state is already digit-valued and clue-consistent. We therefore treat the Zubarev walk as a *finite-move* stochastic optimiser:

- The parameter vector w is the current Sudoku filling $x \in D^{81}$.
- The loss $L(x)$ is the number of conflicting pairs in columns and boxes (equivalently, (1) up to constants in the digit-restricted regime).
- A move ξ is a swap of two non-clue cells within a single row (preserving each row as a permutation of $1, \ldots, 9$).

For a chosen row r and current state x, let $\Xi_r(x)$ be the set of all such swaps, and let $\Delta L(\xi) := L(x \oplus \xi) - L(x)$ be the change in loss after applying swap ξ (where $x \oplus \xi$ denotes "apply the swap"). Replacing the Haar measure in (3) with the counting measure on the finite set $\Xi_r(x)$ gives the discrete sampling rule

$$\Pr(\xi \mid x, \beta_t) = \frac{\exp\bigl(-\beta_t\,\Delta L(\xi)\bigr)}{\sum_{\xi' \in \Xi_r(x)} \exp\bigl(-\beta_t\,\Delta L(\xi')\bigr)}. \tag{4}$$

When $\beta_t \to 0$ this is uniform random swapping; when $\beta_t \to \infty$ it concentrates on the best (most loss-decreasing) swap, recovering a soft version of the greedy stepwise algorithm. In practice we use a simple annealing schedule: β_t increases linearly from β_0 to β_1 over the allotted steps, gradually shifting from exploration to exploitation.

6.3 A short trace on the standard puzzle

Using the same example puzzle as above, the Zubarev walk (seed 0, restart 0, $\beta_0 = 0.5$, $\beta_1 = 6.0$ linear schedule) starts from the same row-consistent initialisation with 56 column+box conflict pairs. Unlike the greedy stepwise method, the walk sometimes accepts loss-increasing or loss-neutral swaps early on; Table 2 shows the first few moves.

For this instance the Zubarev walk reaches a full solution after 10,030 swaps on the same restart. It is far less efficient than the greedy solver on this particular instance—which is exactly the point: once Sudoku has been reduced to a regression loss landscape, even an unreasonably generic training rule can "learn" a satisfying assignment.

Algorithm 2 Zubarev walk over row swaps (heuristic)

Require: Puzzle with clues $\mathcal{C}$, max steps T, restarts R, schedule $\{\beta_t\}_{t=1}^{T}$

```
for r = 1 to R do
    Initialise each row as a permutation of 1, ..., 9 consistent with its clues
    for t = 1 to T do
        if no column/box conflicts then
            return solution
        end if
        Choose a row involved in a conflict
        Enumerate row swaps Ξ_r(x) and compute ΔL(ξ) for each ξ ∈ Ξ_r(x)
        Sample ξ ∈ Ξ_r(x) with probability ∝ exp(−β_t ΔL(ξ))
        Apply swap ξ
    end for
end for
return best assignment found
```

Step	Row	Swap (columns)	Conflicts (before)	(after)
1	5	$2 \leftrightarrow 8$	56	56
2	6	$7 \leftrightarrow 8$	56	56
3	2	$3 \leftrightarrow 7$	56	55
4	8	$2 \leftrightarrow 8$	55	51
5	4	$2 \leftrightarrow 3$	51	50
6	4	$6 \leftrightarrow 7$	50	51
7	2	$7 \leftrightarrow 8$	51	52
8	8	$3 \leftrightarrow 7$	52	49

Table 2: First eight Zubarev-walk swap steps for the example puzzle (seed 0). Rows/columns are 1-indexed.

7 Experiments

We implemented both heuristics above in pure Python (see accompanying code). The prime was set to $p = 11$, but in the digit-restricted regime only the induced indicator behaviour matters.

7.1 Experimental setup

To obtain a quick, reproducible testbed, we generated random puzzles by:

1. sampling a random solved grid; then
2. *carving* it down to a specified clue count by removing entries uniformly at random.

This procedure does *not* enforce uniqueness of the resulting puzzle (it is a fast stress test rather than a curated Sudoku benchmark). Each carved puzzle was then solved with up to $T = 60{,}000$ steps and $R = 15$ random restarts. All results reported below use a fixed RNG seed (123) for reproducibility.

7.2 Results

Table 3 summarises 36 runs (six puzzles at each clue count, for each heuristic). Both solvers succeeded on all instances in this small sample; runtimes are noisy because the puzzles are randomly carved, not difficulty-normalised.

Method	Clues	Puzzles	Solved	Median steps	Median time (s)
Stepwise	36	6	6	538	0.107
Stepwise	30	6	6	2785	0.734
Stepwise	26	6	6	7041	2.027
Zubarev walk	36	6	6	4534	0.800
Zubarev walk	30	6	6	8752	1.987
Zubarev walk	26	6	6	9912	2.929

Table 3: Heuristic performance on randomly carved puzzles (not uniqueness-checked), using $T = 60{,}000$ steps and $R = 15$ restarts. The Zubarev walk uses a linear schedule $\beta_0 = 0.5$ to $\beta_1 = 6.0$.

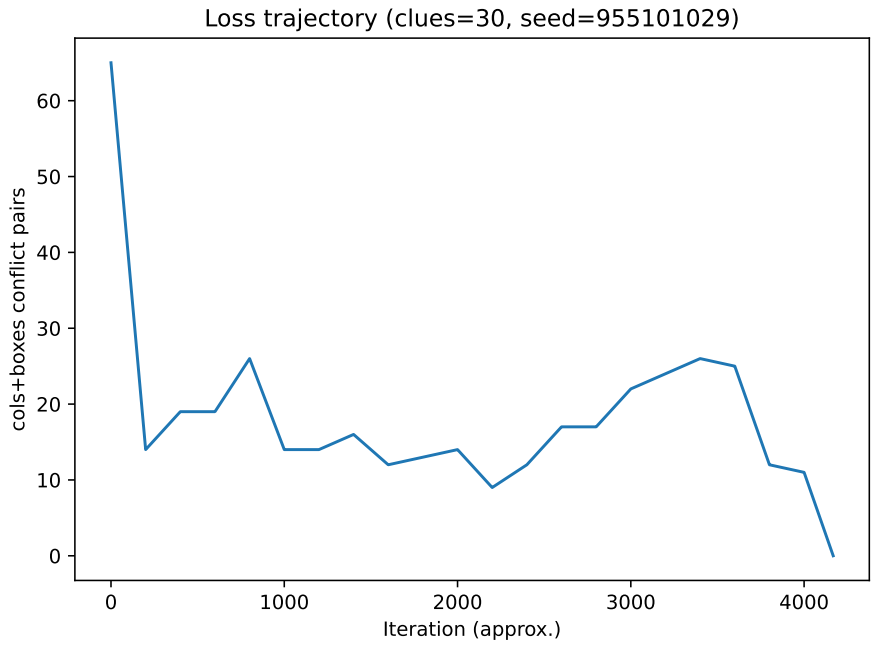

Figure 3: Loss/conflict trajectory for the stepwise row-swap heuristic on the fast-carved 30-clue instance with puzzle seed 955101029 (solve seed 2638744256), from the row-wise random initialisation on restart 0.

Figure 3 shows the loss trajectory for the stepwise row-swap heuristic on the specific fast-carved 30-clue instance with puzzle seed 955101029 (solve seed 2638744256), using the row-wise random initialiser described above, $T = 60{,}000$, and $R = 15$. The plotted run is restart 0, and the loss is the number of conflicting pairs in columns and boxes (equivalently, the objective (1) up to additive and multiplicative constants in the digit-restricted regime).

8 Compared with SAT/CSP encodings

A standard SAT encoding introduces Boolean variables

$$y_{r,c,d} \in \{0, 1\},$$

with the intended meaning that cell (r, c) contains digit d. This gives

$$9 \times 9 \times 9 = 729$$

variables before any clues are applied.

In a direct CNF model, four families of exactly-one constraints are required:

1. one digit per cell (81 constraints);
2. one occurrence of each digit per row (81 constraints);
3. one occurrence of each digit per column (81 constraints);
4. one occurrence of each digit per box (81 constraints).

Flattening each exactly-one constraint on 9 literals into CNF gives one 9-literal clause plus $\binom{9}{2} = 36$ binary exclusion clauses. Hence the expanded SAT input contains

$$324 \text{ long clauses } + \ 324 \cdot 36 = 11{,}664 \text{ binary clauses},$$

for a total of

$$11{,}988 + |\mathcal{C}|$$

clauses after adding one unit clause for each clue in $\mathcal{C}$.

The exact-cover / CSP view uses the same 729 indicator variables, but packages them into 324 structured constraints rather than flattening them all the way to CNF. By contrast, the p-adic regression model works with only 81 digit-valued variables and the objective

$$729 \text{ digit-snapping residuals } + \ |\mathcal{C}| \text{ clue residuals } + \ 810 \text{ pairwise residuals},$$

so the fully expanded regression instance has

$$1{,}539 + |\mathcal{C}|$$

scalar terms. Relative to naive CNF, this is a factor of about

$$\frac{11{,}988}{1{,}539} \approx 7.8$$

fewer expanded constraints, together with a factor of

$$\frac{729}{81} = 9$$

fewer variables. Relative to exact cover, it still uses $9\times$ fewer variables, but more scalar terms than the 324 structured constraints.

One simple information-theoretic proxy for model complexity is the entropy of a uniformly random unconstrained assignment. For the Boolean SAT variables this is

$$H_{\text{SAT}} = \log_2(2^{729}) = 729 \text{ bits},$$

whereas for the p-adic Sudoku variables restricted to $D = \{1, \ldots, 9\}$ it is

$$H_{p\text{-adic}} = \log_2(9^{81}) = 81 \log_2 9 \approx 256.8 \text{ bits}.$$

This is not a theorem about solver runtime, but it does quantify the sense in which the regression model is a compressed parameterisation: it exposes fewer free degrees of freedom. The row-wise permutation heuristic compresses further to $(9!)^9$, with entropy

$$H_{\text{row perm}} = 9 \log_2(9!) \approx 166.2 \text{ bits}$$

before clues, or more generally

$$\sum_{r=1}^{9} \log_2((9 - c_r)!)$$

if row r contains c_r fixed clues. The price of this compression is that the residuals are arithmetic and global rather than local and Boolean.

9 Discussion and extensions

The model is intentionally "the hard way": it embeds Sudoku into a (potentially) NP-hard regression-like optimisation problem rather than exploiting Sudoku structure directly.

Training Sudoku as a regression model. The most absurd aspect of the construction is arguably not the loss itself, but the fact that once we write down (1), we can treat Sudoku completion as a *training problem* and import generic training dynamics from the p-adic regression setting. The Zubarev walk is a particularly clean example: a loss-biased random walk that is not designed for Sudoku in any way, yet still finds valid completions when run over the row-swap move set.

9.1 An alternative sparse encoding: powers-of-two sum constraints (archived)

The objective (1) uses a large collection of pairwise inequality terms: $|\mathcal{P}| = 810$ distinct peer pairs (equivalently $27\binom{9}{2} = 972$ pairs if counted per unit, with overlap). An appealing alternative is to enforce all-different constraints *without* pairwise penalties, by changing the digit representation.

Map digits $d \in \{1, \dots, 9\}$ to powers of two,

$$d \longmapsto 2^{d-1} \in A := \{1, 2, 4, \dots, 256\}.$$

If every cell variable is restricted to A, then the condition "a unit contains each digit exactly once" is equivalent to a *single* sum constraint:

$$\sum_{(r,c)\in g} \beta_{r,c} = 1 + 2 + \cdots + 256 = 2^9 - 1 = 511,$$

for each row/column/box g. This collapses the Sudoku constraints to 27 residuals of the form

$$r_g(\beta) := 511 - \sum_{(r,c)\in g} \beta_{r,c}.$$

For a prime p, one can then study the extremely parameter-sparse p-adic objective

$$L_p(\beta) := \sum_g |r_g(\beta)|_p,$$

optionally together with a (soft or hard) regulariser that encourages $\beta_{r,c} \in A$ and enforces the clues.

This encoding also suggests a "Hensel-ish" lifting perspective: a constraint is satisfied at lift level k when $r_g(\beta) \equiv 0 \pmod{p^k}$ (equivalently $v_p(r_g(\beta)) \geq k$), and a heuristic tries to increase the minimum valuation across groups. Once $p^k > 9 \cdot 256 = 2304$, having all residuals divisible by p^k forces every residual to be zero, yielding an actual integer solution. In archived experiments with simple local-search moves (greedy single-cell changes and simulated annealing), this objective exhibited abundant local minima and did not reliably solve standard benchmark puzzles; however, the representation remains conceptually attractive because it uses so few structural constraints.

Several extensions are immediate:

- **Confusion-aware digit losses.** If clues came from OCR or handwriting rather than clean symbolic input, one could weaken the loss for commonly confused pairs (for example 1 versus 7) instead of treating every wrong digit as equally wrong.
- **Alternative constraint encodings.** Beyond the powers-of-two sum encoding above, one can replace the inequality reward with difference regularisation over the finite set $\{\pm 1, \dots, \pm 8\}$, at the cost of a more complicated objective landscape.

10 Conclusion

This note takes a deliberately contrarian route to a familiar puzzle: it first shows that finite-domain all-different systems admit tidy p-adic regression encodings, and then studies Sudoku as a vivid special case. The construction is mathematically tidy precisely because the p-adic norm turns "different from" into an indicator-like reward once the variables are digit-valued.

As an optimisation method, this is obviously not competitive with mature Sudoku solvers or SAT/CSP technology. But the construction still earns its keep twice over: it gives a genuinely small model—81 digit-valued variables rather than 729 Boolean indicators—and it shows that the same p-adic machinery used in earlier regression work is expressive enough to encode crisp all-different constraint systems in general. Even generic training dynamics such as the Zubarev walk can then recover valid solutions in a concrete case. In that sense the paper is less about Sudoku alone than about the surprising breadth of problems that can be phrased as ultrametric regression landscapes.

References

[1] Gregory Baker, Scott McCallum, and Dirk Pattinson. *Linear Regression in p-adic Metric Spaces. p-Adic Numbers, Ultrametric Analysis and Applications*, 17(4), 2025. Pleiades Publishing.

[2] Gregory Baker. *Expressivity and NP-hardness of p-adic linear regression. p-Adic Numbers, Ultrametric Analysis and Applications*, 18(2), 2026. Pleiades Publishing.

[3] Donald E. Knuth. *Dancing Links. Millennial Perspectives in Computer Science*, 187–214, 2000. Also available as arXiv:cs/0011047.

[4] Helmut Simonis. *Sudoku as a Constraint Problem.* In *Modelling and Reformulating Constraint Satisfaction Problems*, Fourth International Workshop, Sitges (Barcelona), Spain, October 1, 2005.

[5] Inês Lynce and Joël Ouaknine. *Sudoku as a SAT Problem.* In *Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics (ISAIM 2006)*, 2006.

[6] T. Yato and T. Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals*, 86-A(5):1052–1060, 2003.

[7] A. P. Zubarev. *p-Adic Polynomial Regression as Alternative to Neural Network for Approximating p-Adic Functions of Many Variables.* arXiv:2503.23488v2, 2025.